

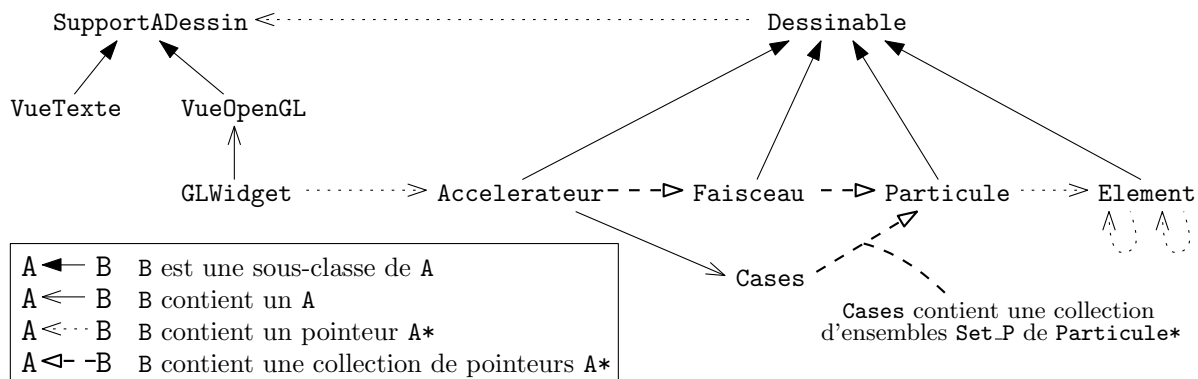
Accélérateur de particules : Conception

Bruno Dular & Hugo Druenne

Printemps 2019

1 Conception générale

1.1 Hiérarchie des classes



Sur la figure ci-dessus est présentée la hiérarchie des classes de notre programme, ainsi que les dépendances entre les différentes classes (possession d'une instance d'une autre classe comme attribut, pointeur vers une instance d'une autre classe, etc.). La notation **A*** signifie un pointeur vers une instance de la classe **A**.

La hiérarchie des différents éléments est détaillée dans la section 1.4 afin de ne pas alourdir le diagramme.

1.2 Classe Vecteur3D

Un **Vecteur3D** représente un vecteur de $\mathbb{R}^3$. Il possède donc trois attributs **x_**, **y_** et **z_**, qui sont les composantes respectives du vecteur.

Un **Vecteur3D** peut être construit en spécifiant les trois composantes. Le constructeur par défaut produit le vecteur nul.

Nous avons implémenté les opérations usuelles sur les vecteurs à l'aide de la *surcharge d'opérateurs* afin de permettre la manipulation des **Vecteur3D**.

1.3 Classe Particule

On modélise une **Particule** au moyen des attributs suivants :

pos_	position	m_kg_	masse en <i>kg</i>
v_	vitesse	q_	charge en <i>C</i>
m_	masse en <i>GeV/c²</i>	F_	force nette subie

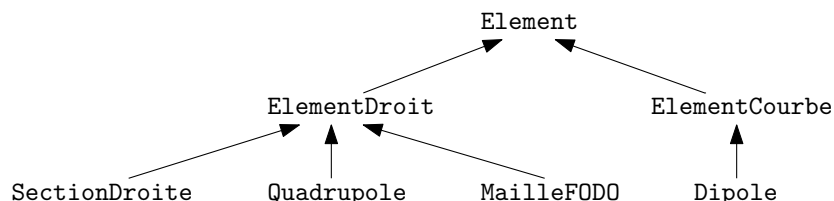
Pour construire une **Particule**, on spécifie sa position, la direction de sa vitesse, son énergie, sa masse et sa charge. La vitesse `v_` est ensuite déduite de la direction et de l'énergie.

Un autre constructeur est fourni et permet de donner la position sous la forme de son *abscisse curviligne* dans l'accélérateur, i.e., un nombre entre 0 et 1 qui donne la position (0 et 1 coïncident avec le début du premier élément, 0,5 correspond à la moitié de l'accélérateur, etc.), ainsi que le sens dans lequel la particule se dirige (sens horaire ou non). La position et la vitesse sont ensuite déduites de ces paramètres au moyen de méthodes spécifiques.

Une **Particule** possède aussi un attribut `element_courant_`, qui est un pointeur vers l'**Element** dans lequel elle se trouve. Ce pointeur est nécessaire pour savoir quelles forces les aimants de l'accélérateur exercent sur la particule. Ce pointeur est un *pointeur à la C* car plusieurs particules peuvent se trouver dans le même élément.

Enfin, afin de permettre l'optimisation *meilleurs voisins*, une **Particule** possède également un attribut `case_courante_`, qui est l'indice de la case dans laquelle elle se trouve. Plus de détails sont donnés dans la section 1.7.

1.4 Classe Element et ses sous-classes



La classe abstraite **Element** possède les attributs suivant :

pos_e_	position d'entrée
pos_s_	position de sortie
r_section_	rayon de la chambre de l'élément
dir_	direction de l'élément
longueur_	longueur
el_suiv_	pointeur vers l'élément suivant
el_prec_	pointeur vers l'élément précédent

Un élément est construit à partir des positions d'entrée et de sortie et du rayon. Les autres attributs sont déduits de ceux-ci. Les pointeurs vers les éléments voisins sont initialisés à `nullptr` et sont correctement initialisés lors de la construction de l'accélérateur. Ce sont aussi des *pointeurs à la C* car des particules pointent aussi vers ces éléments.

Les méthodes principales des **Element** sont :

1. `virtual bool heurte_bord(Particule const& p) const` : retourne `true` ou `false` si une particule donnée a dépassé les parois de l'élément, ou non, respectivement. Cette méthode est virtuelle et définie différemment pour les **ElementDroit** et **ElementCourbe**.

2. `bool passe_au_suivant(Particule& p) const` : si la particule donnée `p` est sortie de l'élément sans être sortie de l'accélérateur, son `element_courant_` prend la valeur de `el_suiv_` ou `el_prec_`, en fonction du côté de sortie. Le cas échéant, la méthode renvoie `true`.
3. `virtual Vecteur3D B(Particule const& p) const` : renvoie le champ magnétique que l'Element exerce sur particule `p`. Cette méthode est virtuelle et redéfinie pour chaque sous-classe d'Element (un élément sans aimants n'exerce pas de champ magnétique, un quadrupôle ou un dipôle en exerce un, etc.)

De `Element` héritent `ElementDroit` et `ElementCourbe`. Un `Elementcourbe` possède les attributs `courbure_`, qui est donné au constructeur, et `centre_`, qui est déduit des autres attributs.

De `ElementDroit` héritent les éléments suivants :

1. `SectionDroite` : élément sans aimants (qui n'exerce donc pas de champ magnétique).
2. `Quadrupole` : élément qui exerce un champ magnétique sur les particules afin de les refocaliser ou défocaliser. Il possède l'attribut `b_`, qui est l'intensité de l'aimant.
3. `MailleFODO` : un élément constitué de deux `SectionDroite` et de deux `Quadrupole`, dont plus de détails sont donnés dans la section 2.1.

De `ElementCourbe` hérite la classe `Dipole`, qui possède l'attribut `Bz_`, l'intensité du champ magnétique vertical. La force exercée par ce champ magnétique vertical fait tourner les particules, afin qu'elles restent sur la trajectoire de l'accélérateur.

1.5 Classe Accelérateur

Un `Accelérateur` possède les attributs suivants :

<code>elements_</code>	collection de pointeurs vers des <code>Element</code>
<code>faisceaux_</code>	collection de pointeurs vers des <code>Faisceau</code>
<code>longueur_</code>	longueur de l'accélérateur
<code>cases_</code>	séparation en cases (voir section 1.7)

La `longueur_` d'un `Accelérateur` est simplement la somme des longueurs des `Elements` qui le composent.

Les `Particules` contenues dans un `Accelérateur` sont regroupées dans des `Faisceaux`. Ceux-ci sont détaillés dans la section 1.6.

Les méthodes principales de la classe `Accelérateur` sont les suivantes :

1. `void evolue(double dt)`, qui fait évoluer les `Particules` contenus dans l'`Accelérateur` en faisant évoluer chacun des `Faisceaux` à la suite.
2. Différentes méthodes permettant de construire un `Accelérateur`, en ajoutant les `Elements` un à un ou au moyen de la méthode `construire_polygone`, détaillée dans la section 2.4
3. `void souder_accelerateur()`, qui permet, une fois les `Elements` ajoutés, d'initialiser leurs attributs `el_suiv_` et `el_prec_`. Cette méthode suppose que les `Elements` ont été ajoutés l'`Accelérateur` dans le bon ordre.

Nous avons implémenté une *abscisse curviligne* le long de la trajectoire de l'`Accelérateur` au moyen de deux méthodes :

1. Vecteur3D `abs_en_pos(double x) const`, qui, étant donné un nombre `x` entre 0 et 1, renvoie la position correspondante sur la trajectoire idéale de l'Accélérateur, avec 0 et 1 le début du premier `Element` ajouté (i.e., `pos_e_` de celui-ci).
2. Abs `pos_en_abs(p_Particule const&) const`, qui, étant donné (un pointeur vers) une `Particule`, donne son abscisse curviligne.

Ces méthodes se servent de méthodes homonymes définies pour les `Elements`.

1.6 Classe Faisceau

Un `Faisceau` permet de manipuler une collection de `Particules`, afin de ne pas devoir manipuler celles-ci individuellement. En fait, on considère des *macro-particules*, c'est-à-dire, par exemple, qu'on ne simule pas 1000 particules, mais 100 macro-particules 10 fois plus massives.

Un `Faisceau` est modélisé au moyen des attributs suivants :

1. `lambda_`, qui est le coefficient déterminant la *taille* des macro-particules, i.e., le nombre de particules réelles simulées par une macro-particule.
2. `particules_`, qui est un vecteur de pointeurs vers les `Particules` formant le `Faisceau`.

Le constructeur de `Faisceau` prend une `Particule`, utilisée comme modèle pour créer les macro-particules constituant le `Faisceau`, le coefficient `lambda_`, ainsi que des paramètres spécifiant la position et le sens de déplacement du `Faisceau`. Il demande aussi un booléen spécifiant si la répartition des `Particules` au sein du `Faisceau` se fait selon une distribution normale ou linéaire. Plus de détails sont donnés dans la section 2.3.

Les deux méthodes principales de la classe `Faisceau` sont les suivantes :

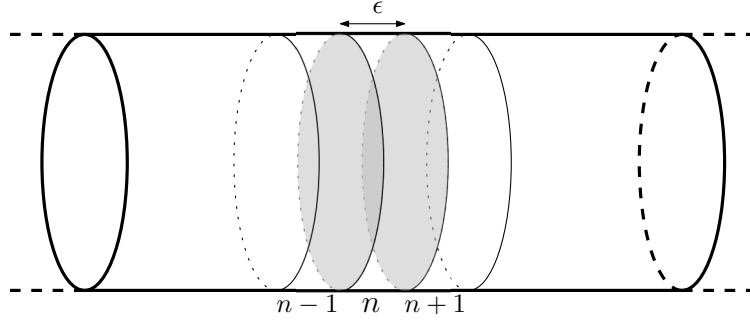
1. `void initialiser_particules(Accelerateur const& acc, Cases& cases)`, qui initialise les pointeurs `element_courant_` de chacune des `Particules` constituant le `Faisceau`. Cette méthode initialise aussi la configuration des `Cases`, qui est développée en détail dans la section 1.7.
2. `void evolue(Accelerateur const& acc, Cases& cases, double dt)` est la méthode qui permet l'évolution du `Faisceau` au sein de l'Accélérateur. Etant donné un petit pas de temps `dt`, elle boucle sur toutes les `Particules` du `Faisceau`. Pour chaque `Particule`, cette méthode ajoute les forces qu'elle subit pour la faire se déplacer et ensuite met à jour l'`element_courant_` de la `Particule`, ou, si elle sort de l'accélérateur, la supprime.

1.7 Interactions inter-particules et classe Cases

L'évolution d'une `Particule` est influencée par les forces que les autres `Particules` exercent sur elle. Mais calculer les forces exercées par *toutes* les autres `Particules` prend plus de temps. De plus, la force exercée par les `Particules` éloignées est négligeable. C'est pour optimiser ce calcul que la classe `Cases` a été implémentée.

La configuration en `Cases` est représentée au moyen des attributs suivants :

<code>cases_</code>	collection d'ensembles de <code>Particule*</code>
<code>eps_</code>	longueur d'une tranche
<code>nombre_</code>	nombre de tranches



L'Accélérateur est découpé en `nombre_tranches` de longueur `eps_`, comme illustré sur la figure ci-dessus.

Une *tranche* est en fait simplement un `unordered_set` de `Particule*` : un ensemble de pointeurs vers les `Particules` qui se trouvent dans cette tranche.

Une `Particule` donnée, d'abscisse curviligne x , est dans la $\lfloor \frac{x}{\epsilon} \rfloor$ -ème tranche (où $\lfloor a \rfloor$ dénote la partie entière de a).

Lors du calcul des forces inter-particulaires, une `Particule` se trouvant dans la n -ème tranche ne prend en compte que les `Particules` se trouvant dans les tranches $n - 1$, n et $n + 1$.

1.8 Classes SupportADessin, Dessinable et graphisme

Nous avons implémenté l'affichage de l'accélérateur au moyen de deux classes : `SupportADessin` et `Dessinable`. Toutes les classes détaillées dans les sections précédentes, à l'exception de `Vecteur3D`, modélisent des objets que l'on souhaite dessiner, soit sous format texte, soit dans une fenêtre graphique. Ce sont donc des sous-classes de `Dessinable`, une classe abstraite générique pour tout objet *dessinable*.

Ces objets `Dessinables` sont affichés sur un `SupportADessin` dont ils possèdent l'adresse en attribut. La classe `SupportADessin` est aussi une classe abstraite qui modélise un *affichage*.

De `SupportADessin` héritent les classes `VueTexte` et `VueOpenGL`. La première permet l'affichage *console*, tandis que la seconde permet l'affichage sur une *fenêtre graphique*. C'est dans ces sous-classes *concrètes* que l'on peut définir les méthodes permettant de dessiner les `Dessinables`.

2 Extensions

2.1 L'élément MailleFODO

Comme mentionné dans la section 1.4, une `MailleFODO` est un `ElementDroit` qui est constitué de deux *quadrupôles* et de deux *section droites*. Nous avons choisi l'implémentation simple, c'est-à-dire qu'une `MailleFODO` ne contient pas ces éléments tels quels, mais le champ magnétique qu'elle exerce sur une `Particule` dépend de sa position. Si celle-ci se trouve dans les parties *section droite*, le champ sera nul, sinon, le champ correspondra au champ magnétique engendré par un *quadrupôle*.

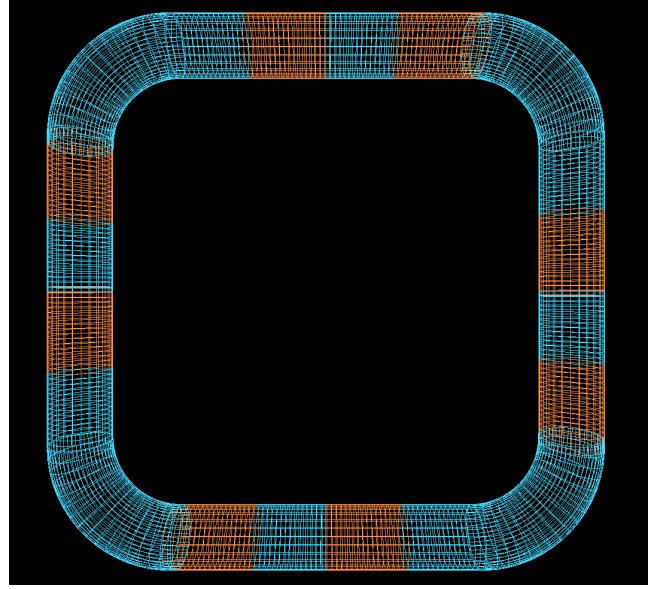
Le constructeur de `MailleFODO` prend donc en paramètre la proportion de la longueur des *quadrupôles* par rapport à longueur totale. Cette classe est aussi dotée de *getters* correspondant aux positions d'entrée et de sorties des différentes parties de la `MailleFODO`.

2.2 Affichage des éléments

Pour rendre notre accélérateur plus *esthétique* et *réel*, nous avons décidé d’afficher les éléments complètement, comme le montre la figure ci-contre. Cela nécessite les deux méthodes de `VueOpenGL` suivantes :

1. `dessineCylindre`, qui dessine un tube cylindrique étant donné les positions d’entrée et de sorties, et le rayon, du cylindre.
2. `dessineTore`, qui dessine une section de *tore* à partir des positions d’entrée et de sorties, du centre du tore et du rayon du tube.

Ces méthodes prennent aussi en paramètres une couleur (trois double, *RVB*). Nous avons décidé de dessiner en *bleu* les éléments qui exercent un champ magnétique, et en *rouge* les autres.



2.3 Faisceaux améliorés et affichage

2.3.1 Couleur des particules

Dans notre affichage, la couleur d’une particule dépend de la force qu’elle subit. Plus celle-ci est forte, plus la particule est blanche. Cela permet de bien visualiser les forces exercées par les éléments magnétiques.

2.3.2 Distribution normale pour les faisceaux

Lors de la construction d’un *Faisceau*, on a le choix entre une distribution *linéaire* ou *normale*. La distribution *linéaire* répartit simplement les particules le long de la trajectoire idéale de manière uniforme. La distribution *normale*, quant à elle, les répartit selon une loi *Gaussienne* dans les trois dimensions (Figure 1).

2.3.3 Affichage des paramètres des ellipses de phase des faisceaux

A chaque *Faisceau* est associée une *ellipse de phases*, dont les coefficients dépendent de la distribution des particules au sein du faisceau. Nous représentons ces ellipses dans des fenêtres graphiques additionnelles, une pour chaque faisceau (Figure 2). Cela permet de visualiser, par exemple, la variation de répartition lors du passage par un *dipôle*.

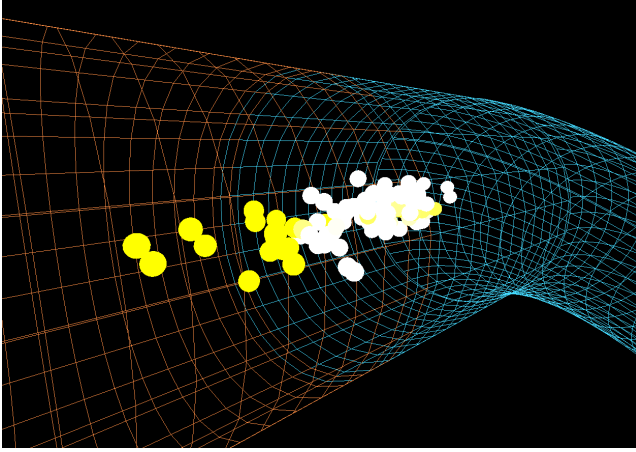


FIGURE 1 – Faisceau de particules

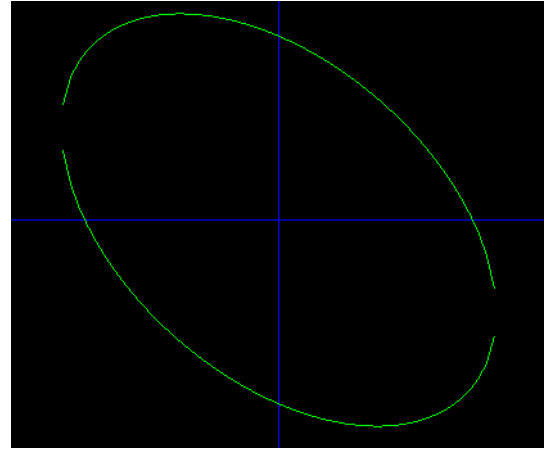


FIGURE 2 – Ellipse de phases

2.4 Accélérateurs polygonaux et optimisation du rayon

En réalité, un accélérateur de particules n'est pas carré, mais circulaire. Et qu'est-ce qu'il y a entre un carré et un cercle? Des polygones réguliers!

Nous avons muni la classe `Accelerateur` d'une méthode

```
void construire_polygone(size_t n, double R);
```

qui, étant donnés n et R , construit un accélérateur de la forme d'un polygone régulier à n côtés, et de rayon R (Figures 3-5).

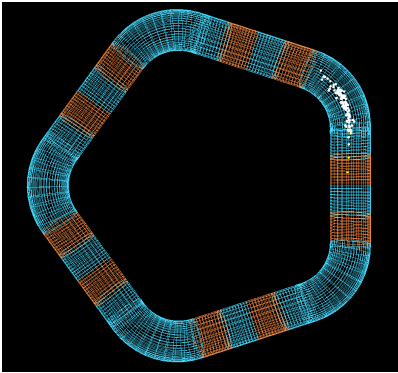


FIGURE 3 – Pentagone

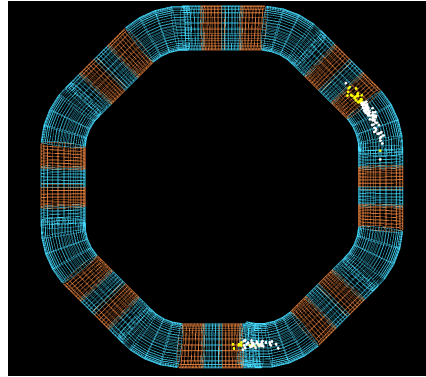


FIGURE 4 – Octogone

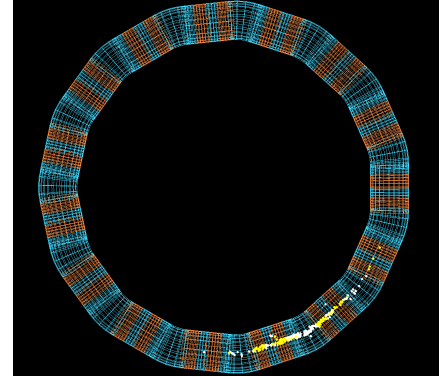


FIGURE 5 – 15-gone

Afin de voir l'influence de la forme de l'accélérateur sur l'évolution des particules, nous avons fait tourner des simulations pour un accélérateur à n côtés et de rayon R , pour $3 \leq n \leq 11$ et $R \in [1.2, 3.9]$ (avec un pas de 0.1). De cette analyse on peut déduire que la stabilité décroît lorsque le rayon augmente, et croît avec le nombre de côtés. Cela suggère que la forme la plus stable est le cercle.

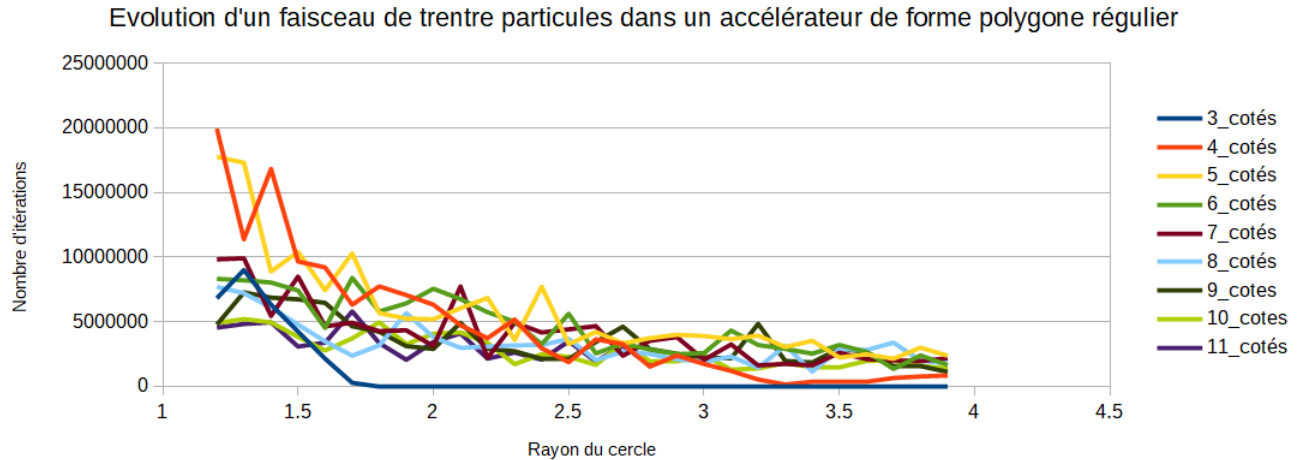


FIGURE 6 – Influence du nombre de côtés et du rayon sur la stabilité des particules

2.5 Interface utilisateur

Afin de faciliter l'utilisation de notre simulateur, nous avons mis au point une interface utilisateur. Celle-ci permet de modifier la forme de l'accélérateur et d'y introduire des faisceaux sans avoir à modifier le code source. Cette interface est simplement présentée sous forme de questions posées à l'utilisateur, qui peut alors saisir les paramètres de son choix.

```
Bienvenue dans ce simulateur d'accélérateur de particules !
Pour commencer, construisons un accélérateur.
Nombre de côtés : (  $\geq 3$  ) 15
Rayon de l'accélérateur : (  $\geq 1$  ) 3
Ajouter un faisceau de particules ? o pour OUI, n pour NON o
Sens horaire ? o pour OUI, n pour NON o
Faisceau centré en : (  $\geq 0$  ) 0
Nombre de particules : (  $\geq 1$  ) 100
Facteur de macro-particules : (  $\geq 1$  ) 1
Etendue du faisceau : (  $\geq 0.05$  ) 0.2
Distribution normale (OUI) ou linéaire (NON) : o pour OUI, n pour NON o
```

3 Améliorations possibles

Grâce à la modularité du code et à l'approche *orienté-objet*, il serait possible d'ajouter de multiples extensions sans devoir modifier la structure du code. Voici quelques améliorations possibles :

1. Implémenter une interface utilisateur graphique, avec des boutons et des curseurs.
2. Permettre à l'utilisateur d'ajouter des faisceaux pendant la simulation.
3. Créer des accélérateurs de forme arbitraire (en huit, avec looping, etc.).
4. Comme proposé dans les consignes, ajouter des éléments dotés d'un champ électrique (cavité radio-fréquence).
5. Implémenter les collisions entre particules.